# Python 3.15 Feature Tour

A hands-on tour of Python 3.15: install it on Windows with uv, run an 11-test smoke script, and read the code and output for each feature to see what changed.

### Python 3.15 Tutorial: Install, Test, Understand

Install 3.15.0 on Windows, run a feature smoke test, and explain every result

*Windows + uv + PowerShell*

## 1 Overview

Python 3.15 makes UTF-8 the default file encoding on Windows, adds explicit lazy imports, allows unpacking inside comprehensions, and introduces a `sentinel` builtin, a `frozendict`, and a sampling profiler. This lab installs 3.15 on Windows, runs one script that tests eleven of those features, and explains the code and the output behind each result.

> **Sequence:** Install the interpreter, confirm the version, run the script, then read the feature sections to see why each test passes.

| Feature | What it does | Test name |
|---|---|---|
| UTF-8 default | `open()` without `encoding=` reads and writes UTF-8 on Windows | utf8_default_open |
| Lazy import | `lazy import json` delays loading until first use | lazy_import |
| Comprehension unpacking | `[*sub for sub in nested]` flattens one level | unpacking_in_comprehension |
| sentinel | Builtin for unique marker values with a readable repr | sentinel |
| frozendict | Immutable, hashable dictionary | frozendict |
| Counter ^ | Symmetric difference of counts | counter_symmetric_difference |
| re.prefixmatch | A clearer name for `re.match` | re_prefixmatch |
| iter_graphemes | Loop over the characters you actually see | unicodedata_iter_graphemes |
| pprint expand | JSON-style layout for pretty-printed data | pprint_expand |
| Error suggestions | `AttributeError` hints, for example push suggests append | better_error_message |
| Sampling profiler | Low-overhead profiler via `python -m profiling.sampling` | sampling_profiler_available |

© 2026 David Grunwald. All rights reserved.

## 2 Install Python 3.15 on Windows

`winget` was not available on this machine, so the lab uses the standalone uv installer. uv downloads and manages Python builds and creates the virtual environment.

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
# close and reopen PowerShell, then:
uv python install 3.15
uv venv --python 3.15 .venv315
.\.venv315\Scripts\Activate.ps1
python -c "import sys; print(sys.version)"
```

Expected output from the last command:

```
3.15.0 (main, Oct 14 2026, 01:12:50) [MSC v.1951 64 bit (AMD64)]
```

> **Seen in practice:** The first venv reported `3.15.0a5`, an alpha from January 2026, and the script stopped with `SyntaxError: iterable unpacking cannot be used in comprehension`. A second, older uv installed with pip in `C:\python3_13\Scripts` came first on PATH. Calling the standalone copy by its full path found the 3.15.0 build:

```
& "$env:USERPROFILE\.local\bin\uv.exe" venv --python 3.15 .venv315 --clear
```

> **Transferable habit:** Check `sys.version` before debugging. A syntax error on a documented feature usually means an older build, not a bug in your code.

## 3 How the test script works

The script defines a `test` decorator that runs each function the moment it is defined, prints PASS or FAIL, and prints the traceback on failure. One failing feature therefore never hides the others. At the end it prints a count and exits with status 1 if anything failed.

```
def test(fn):
    name = fn.__name__.removeprefix("test_")
    try:
        outcome = fn()
        status = "SKIP" if outcome == "skip" else "PASS"
    except Exception:
        status = "FAIL"
        traceback.print_exc()
    results.append((name, status))
    print(f"[{status}] {name}")
    return fn
```

Two features are new syntax, so on an older Python they would stop the whole file at compile time, before any test ran. The script hands that syntax to `exec()` as a string, which turns a compile-time failure into a failure of that single test:

© 2026 David Grunwald. All rights reserved.

```
ns = {"nested": [[1, 2], [3], [4, 5]]}
exec("result = [*sub for sub in nested]", ns)
assert ns["result"] == [1, 2, 3, 4, 5]
```

# 4 Feature walkthrough

## UTF-8 by default

Before 3.15, `open()` on Windows used the locale code page unless you passed `encoding=`, so text written on one machine could read differently on another. The test writes accented and Japanese text with no encoding argument, reads the raw bytes back, and checks they equal the UTF-8 encoding.

```
with open(path, "w") as f:          # no encoding= on purpose
    f.write(text)
with open(path, "rb") as f:
    assert f.read() == text.encode("utf-8")
```

Output: `[PASS] utf8_default_open`

## Explicit lazy imports

`lazy import json` registers the name but does not load the module until the name is first used. For command-line tools this can shorten startup time. The test checks `sys.modules` before and after the first use. It runs through `exec()` because `lazy import` is syntax.

```
lazy import json
loaded_before = 'json' in sys.modules    # False: nothing loaded yet
json.dumps({})                           # first use triggers the import
loaded_after = 'json' in sys.modules     # True
```

Output: `[PASS] lazy_import`

## Unpacking in comprehensions

A star in front of the loop variable splices each sub-list into the result, replacing the two-loop flattening idiom.

```
nested = [[1, 2], [3], [4, 5]]
[*sub for sub in nested]                 # [1, 2, 3, 4, 5]
[x for sub in nested for x in sub]       # the older, two-loop spelling
```

Output: `[PASS] unpacking_in_comprehension`

## sentinel

A sentinel is a unique marker such as "no value was passed". Plain `object()` works but prints as `<object object at 0x...>`. `sentinel("MISSING")` has a readable representation containing its name. The test checks that the object is itself and that its repr includes `MISSING`.

```
MISSING = sentinel("MISSING")
assert MISSING is MISSING
assert "MISSING" in repr(MISSING)
```

© 2026 David Grunwald. All rights reserved.

Output: `[PASS] sentinel`

## frozendict

An immutable dictionary. Because it cannot change, it is hashable, so it can be a dictionary key or a set member. The test checks lookup, hashing, and that assignment raises `TypeError`.

```
fd = frozendict(a=1, b=2)
assert fd["a"] == 1
assert isinstance(hash(fd), int)         # hashable, so usable as a dict key
fd["c"] = 3                              # raises TypeError: immutable
```

Output: `[PASS] frozendict`

## Counter symmetric difference

The `^` operator keeps, for each key, the absolute difference between the two counts. Here `a` is |3 - 1| = 2, `b` is |1 - 0| = 1, and `c` is |0 - 2| = 2.

```
>>> from collections import Counter
>>> Counter(a=3, b=1) ^ Counter(a=1, c=2)
Counter({'a': 2, 'c': 2, 'b': 1})
```

Output: `[PASS] counter_symmetric_difference`

## re.prefixmatch

`re.prefixmatch` is a clearer name for `re.match`: the pattern must match at the start of the string, not anywhere inside it.

```
re.prefixmatch(r"\d+", "123abc").group()   # '123'
re.prefixmatch(r"\d+", "abc123")           # None: only matches at the start
```

Output: `[PASS] re_prefixmatch`

## unicodedata.iter_graphemes

Python counts code points, but people see graphemes. The test string is the letter e plus a combining accent (U+0301), a flag emoji, and a family emoji, which is far more than three code points but exactly three visible characters. `iter_graphemes` yields the visible units.

```
s = "e\u0301" + flag_emoji + family_emoji   # three things you see on screen
parts = list(unicodedata.iter_graphemes(s))
assert len(parts) == 3
```

Output: `[PASS] unicodedata_iter_graphemes`

## pprint with expand=True

`expand=True` puts a newline after each opening bracket and indents the contents, like pretty-printed JSON. It only applies to structures that do not fit within `width`, so the test sets `width=20` to force a break. A higher `indent` such as 3 reads best.

© 2026 David Grunwald. All rights reserved.

```
data = {"a": [1, 2, 3], "b": {"c": 4}}
plain = pprint.pformat(data, width=20)
expanded = pprint.pformat(data, width=20, expand=True, indent=3)
```

Output (repr of the two strings):

```
plain:    "{'a': [1, 2, 3],\n 'b': {'c': 4}}"
expanded: "{\n   'a': [1, 2, 3],\n   'b': {'c': 4},\n}"
```

Printed, the expanded version looks like this:

```
{
   'a': [1, 2, 3],
   'b': {'c': 4},
}
```

> **Seen in practice:** The first version of the test used the default width of 80. Both calls returned identical one-line output because the data already fit, which looked like a bug. The docs state that objects stay on one line when they fit, and their own example uses `width=41`.

## Better error messages

Python 3.15 recognizes habits carried over from other languages and suggests the Python name. The suggestion is added when the traceback is formatted, so the test builds the text with `traceback.format_exception` and looks for `append`.

```
try:
    [].push(1)
except AttributeError as e:
    text = "".join(traceback.format_exception(e))
    assert "append" in text
```

| You typed | Python suggests |
|---|---|
| `[].push(1)` | `append` |
| `"x".toUpperCase()` | `upper` |
| `"x".trim()` | `strip` |
| `d.keySet()` | `keys` |

Output: `[PASS] better_error_message`

## Sampling profiler

`profiling.sampling` samples a running program instead of tracing every call, so overhead is low, and it can attach to a process that is already running. The test only confirms the module exists by running `--help` and checking for exit code 0. Try the live view on a real script:

© 2026 David Grunwald. All rights reserved.

```
python -m profiling.sampling --help
python -m profiling.sampling --live script.py
```

Output: `[PASS] sampling_profiler_available`

## 5 Reading the full output

```
(.venv315) PS C:\Users\degru> python py315_features_test.py
[PASS] utf8_default_open
[PASS] lazy_import
[PASS] unpacking_in_comprehension
[PASS] sentinel
[PASS] frozendict
[PASS] counter_symmetric_difference
[PASS] re_prefixmatch
[PASS] unicodedata_iter_graphemes
   plain:    "{'a': [1, 2, 3],\n 'b': {'c': 4}}"
   expanded: "{\n   'a': [1, 2, 3],\n   'b': {'c': 4},\n}"
[PASS] pprint_expand
[PASS] better_error_message
[PASS] sampling_profiler_available
Python 3.15.0 (main, Oct 14 2026, 01:12:50) [MSC v.1951 64 bit (AMD64)]
11/11 passed
```

Each `[PASS]` line is one test. The two indented lines are `print()` calls inside the pprint test, kept so you can see what `expand=True` produced. The last two lines are the interpreter version and the tally. In PowerShell, `$LASTEXITCODE` shows 0 when everything passed and 1 otherwise.

## 6 Corrections from the run

| Symptom | Cause | Fix |
|---|---|---|
| SyntaxError on `[*sub for sub in nested]` | venv was the 3.15.0a5 alpha | Recreate the venv with standalone uv; confirm 3.15.0 |
| `uv self update` refused | Two uv copies; the pip-installed one was first on PATH | Call standalone uv by full path, or `pip uninstall uv` |
| Script died before any test ran | New syntax compiled with the whole file | Wrap the syntax in `exec()` |
| `pprint_expand` failed | Sample dict fit within 80 columns | Use `width=20` to force wrapping |
| `winget` not recognized | App Installer not present | Use the uv PowerShell installer |

*Python 3.15 · uv · Windows · lazy imports · frozendict · pprint · sampling profiler*

*Python 3.15 Feature Tour · v1.0 · 2026*

*© 2026 David Grunwald. All rights reserved.*